

Guía de Estudio — Tema 12: Manejo de Excepciones

Curso: Paradigmas y Lenguajes de Programación **Institución:** Universidad Nacional de Tierra del Fuego — Instituto IDEI **Ciclo lectivo:** 2026 · Módulo IX · Semana 12 **Tema:** 12 — Manejo de Excepciones **Duración de la clase:** 120 minutos **Lenguaje principal:** TypeScript **Lenguajes de contraste:** Go, Kotlin, Rust **Bibliografía principal:** Sebesta, *Concepts of Programming Languages*, Cap. 14, pp. 611–646 **Bibliografía auxiliar:** Gabbrielli & Martini §7.3.1; Louden & Lambert §9.5 **Pipeline:** v3 · Nivel de densidad: 2 · Corregido: 2026-06-28

1. Introducción

Esta guía es tu material de estudio autónomo para el Tema 12. Está construida sobre la clase dictada (filminas F-00 a F-17) y profundiza cada concepto con la bibliografía de respaldo. Si un alumno puede estudiarlo solo, lo hicimos bien.

El manejo de excepciones es una **decisión de diseño de lenguaje** — no un detalle de sintaxis. Sebesta dedica un capítulo entero a este tema porque afecta la legibilidad, la correctitud y la composabilidad del código. En esta guía vas a ver por qué TypeScript, Go, Kotlin y Rust responden de maneras distintas a las mismas preguntas de diseño, y por qué eso importa cuando escribís código real — especialmente en programación agéntica.

Cómo usar esta guía:

1. Leé la sección de **Conceptos previos** para verificar que tenés la base del Tema 11.
 2. Recorré el **Desarrollo teórico** en orden — cada sección referencia la filmina correspondiente ([F-XX]) y la cita bibliográfica.
 3. Hacé los **Ejemplos trabajados** con lápiz y papel antes de mirar la resolución.
 4. Usá los **Puntos clave** como cheat-sheet de repaso rápido.
 5. Resolvé la **Autoevaluación** — las respuestas están ocultas en bloques desplegados.
 6. Consultá el **Glosario** cuando un término no quede claro.
-

2. Objetivos de Aprendizaje

Al finalizar el estudio de esta guía vas a poder:

1. **Definir** qué es una excepción, cómo se levanta y cómo se liga a un handler — siguiendo la terminología de Sebesta Cap. 14.
2. **Distinguir** el modelo de terminación del de reanudación y justificar por qué el primero domina los lenguajes modernos.
3. **Explicar** el mecanismo de propagación de excepciones a lo largo del call stack.
4. **Implementar** manejo de excepciones en TypeScript usando `try / catch / finally` y excepciones user-defined.
5. **Implementar** el enfoque funcional con `Result<T,E>` en TypeScript.
6. **Comparar** cómo Go, Kotlin y Rust abordan el manejo de errores, contrastando con TypeScript.
7. **Evaluar** las ventajas del enfoque funcional vs. imperativo según el contexto de uso.
8. **Identificar** por qué el manejo de excepciones es crítico en la programación agéntica.

Fuente: `diseno.md` — Objetivos de Aprendizaje, aprobado 2026-06-28.

3. Conceptos Previos

Antes de profundizar en excepciones, necesitás tener clara la base del Tema 11. Esta guía **no re-explica** esos conceptos — solo verifica que los manejá.

3.1. Estructuras de control (Tema 11)

Las excepciones son un mecanismo de **control de flujo no lineal**. Mientras que `if`, `while` y `for` son control de flujo lineal (el código ejecuta de arriba hacia abajo, con bifurcaciones locales), las excepciones pueden **saltar** desde un punto profundo del call stack hasta un handler que está varios niveles arriba. Esto es lo que las hace potentes — y también lo que las hace difíciles de razonar.

Conceptos del Tema 11 que vas a necesitar:

- Estructuras de control secuenciales, condicionales e iterativas.
- Diferencia entre control de flujo lineal y no lineal.
- Concepto de call stack (pila de llamadas a funciones).

3.2. Mónadas en TypeScript (Tema 5)

El enfoque funcional `Result<T,E>` que veremos en la sección §4.5 es la **mónada Either**. Si hiciste el Tema 5, ya conocés `map` y `flatMap`. Si no lo hiciste, no te preocupes — la guía explica `Result` desde cero, pero saber mónadas te va a dar una ventaja conceptual.

Conexión curricular: `Result<T,E>` es la mónada Either. En Haskell es `Either a b`. En fp-ts es `Either`. La idea es la misma. [Conexión: Tema 5 — Mónadas]

3.3. Tipos algebraicos y union types en TypeScript

TypeScript usa **union types** (`A | B`) para representar valores que pueden ser de varios tipos. El patrón `Result<T,E>` se implementa como una *discriminated union* — un union type donde un campo común (`ok: boolean`) le dice al compilador qué variante es. Si no estás familiarizado con discriminated unions, el código de la sección §4.5 te va a mostrar el patrón en contexto.

4. Desarrollo Teórico

4.1. ¿Qué es una excepción?

Filmina de referencia: [F-03]

Una excepción es un evento anómalo que ocurre durante la ejecución (runtime) y que el código normal no puede manejar en línea. La definición canónica es de Sebesta:

“Una **excepción** es cualquier evento inusual, ya sea erróneo o no, detectable por hardware o software, que puede requerir un procesamiento especial.”
— [Sebesta, *Concepts of Programming Languages*, Cap. 14 §14.1, p. 611] (traducción)

 **Punto clave:** Sebesta dice “erroneous or not” — **no toda excepción es un error**. Un archivo no encontrado no es un bug del programador; es una condición esperable pero infrecuente del entorno. La distinción es importante:

Situación	Tipo	Cómo se resuelve
Error de lógica en el código	Bug	Debugging
Condición inesperada en runtime	Excepción	Handler
Archivo no encontrado, red caída, input inválido	Excepción	Handler

Los cuatro componentes

Sebesta identifica cuatro elementos que componen el mecanismo de excepciones [Sebesta, Cap. 14 §14.1, p. 611]:

1. **Exception:** el evento en sí — detectado en runtime.
2. **Exception handler:** el código que responde al evento.
3. **Raise / throw:** disparar la excepción (lanzarla).
4. **Catch:** capturarla en el handler.

Gabrielli & Martini añaden una observación importante: una excepción no es un evento anónimo — tiene un nombre, y a menudo es un valor de uno de los tipos del lenguaje:

“Una excepción no es un evento anónimo. Tiene un nombre (a menudo, más bien, es un valor en uno de los tipos del lenguaje) que se menciona explícitamente en el constructo throw.”

— [Gabbrielli & Martini, *Programming Languages*, Cap. 7 §7.3.1, p. 282] (traducción)

Esto significa que cuando escribís `throw new HttpError(404, "Not Found")`, la excepción no es un evento difuso — es un **valor tipado** que viaja por el call stack hasta encontrar un handler que sepa qué hacer con ese tipo específico.

4.2. El problema antes de las excepciones

Filmina de referencia: [F-04]

Para entender por qué las excepciones existen, hay que entender qué había antes. Louden & Lambert describen los mecanismos pre-try/catch [Louden & Lambert, *Programming Languages*, Cap. 9 §9.5, p. 406]:

Patrón 1: Códigos de retorno ©

```
// C - sin excepciones: flags de error + errno global
FILE *f = fopen("data.txt", "r");
if (f == NULL) {
    fprintf(stderr, "Error: %s\n", strerror(errno));
    return -1; // propagar manualmente por cada nivel
}
// el llamador también debe chequear el -1...
```

Patrón 2: Callback de error ©

```
typedef void (*ErrorHandler)(ErrorKind);

void processFile(const char *path, ErrorHandler onError) {
    // si algo falla: llamar onError en lugar de retornar -1
    // el handler es inyectado por el caller
}
```

Louden describe este patrón como pasar un procedimiento de manejo de errores como parámetro [Louden & Lambert, Cap. 9 §9.5, p. 425]:

```
enum ErrorKind {OutOfInput, BadChar, Normal};
typedef void (*ErrorProc)(ErrorKind);

void handler(ErrorKind error) { /* ... */ }

unsigned getNumber(ErrorProc proc) { /* ... */ }
```

Problemas de estos enfoques

- 1. **Fácil olvidar chequear** el código de retorno → el error se silencia y el programa sigue con datos corruptos.
 - 2. El código de manejo de errores **mezcla** con la lógica principal — difícil de leer.
 - 3. `errno` es **global** — no es thread-safe, se sobrescribe entre llamadas.
 - 4. El callback solo cubre **un** tipo de error por función.
- Estos problemas motivaron la invención del `try/catch`. La idea central: **separar** el código de lógica principal del código de manejo de errores.

Conexión con Go: Go volvió a errores como valores, pero con un sistema de tipos más fuerte. Lo vemos en la sección §4.6.

4.3. Historia: de PL/I a TypeScript

Filmina de referencia: [F-05]

Año	Lenguaje	Aporte
~1964	PL/I	Primer mecanismo formal — modelo de reanudación
1983	Ada	Modelo de terminación — se convierte en el estándar
1985	C++	<code>try/catch/throw</code> — excepciones como objetos
1995	Java	Jerarquía <code>Throwable</code> + checked exceptions
2009	Go	Rechaza excepciones — <code>(T, error)</code> como valores
2011	Kotlin	Sin checked exceptions + <code>sealed class</code> para errores tipados
2015	Rust	Sin excepciones en runtime — <code>Result<T,E></code> obligatorio
2012+	TypeScript	<code>try/catch</code> heredado de JS + tipado con union types
2022	ES2022	<code>Error.cause</code> estándar: encadenamiento nativo de errores
2023	Go 1.20	<code>errors.Join</code> : un error puede envolver múltiples errores

Tendencia clave: los lenguajes más recientes prefieren errores como **valores** sobre excepciones como **flujo de control**. Esto no es accidental — lo vamos a entender en las próximas secciones.

4.4. Terminación vs. Reanudación

Filmina de referencia: [F-06]

Esta es la decisión de diseño más importante del capítulo. Cuando una excepción se lanza, ¿qué pasa con el código que la generó? Hay dos modelos:

Aspecto	Terminación	Reanudación
¿Qué pasa al lanzar?	El scope que generó la excepción termina	El scope que generó la excepción se pausa
¿Dónde sigue?	En el handler — nunca vuelve al raise	El handler ejecuta → vuelve al punto del raise
Lenguajes	Java, C++, C#, TypeScript, Kotlin, Go ¹ , Rust ¹ , Python	PL/I (legacy), algunos LISP
Ventaja	Simple, predecible, stack unwinding claro	Permite “corrección” y retry in-place
Desventaja	No se puede continuar donde se cortó	El handler debe conocer el contexto interno del caller

¹ Go y Rust no tienen excepciones en el sentido clásico: usan valores de retorno (`error` , `Result<T, E>`). Se incluyen aquí porque su modelo de propagación (`?` , retorno explícito) sigue la semántica de **terminación** — no hay vuelta al punto de falla.

Sebesta es claro sobre por qué la terminación domina:

“La **terminación** es obviamente el más simple de los dos modelos y es el modelo utilizado en la mayoría de los lenguajes contemporáneos.”

— [Sebesta, *Concepts of Programming Languages*, Cap. 14 §14.2.3, p. 612] (traducción)

Gabbrielli & Martini complementan desde la perspectiva de implementación:

“Esta forma de operar se denomina ‘manejo con terminación’ — el constructo donde se determina la excepción queda **terminado**.”

— [Gabbrielli & Martini, *Programming Languages*, Cap. 7 §7.3.1, p. 282] (traducción)

¿Por qué casi nadie usa reanudación?

Si el handler ejecuta y vuelve al punto del `raise` , necesita **conocer el estado interno** del código que lanzó la excepción para poder “arreglarlo” y retomar. Eso es un acoplamiento muy alto entre el handler y el código que falló. En la práctica, esto hace que el modelo de reanudación sea difícil de usar correctamente.

Sebesta lo explica así: la opción de volver al statement que lanzó la excepción “puede parecer buena, pero en el caso de un error, es útil solo si el handler puede modificar los valores u operaciones que causaron que la excepción se levantara. De lo contrario, la excepción simplemente se relanzará” [Sebesta, Cap. 14 §14.2.3, p. 612].

4.5. Las 8 preguntas de diseño de lenguaje

Filmina de referencia: [F-07]

Sebesta enumera las decisiones de diseño que **todo** lenguaje debe tomar respecto al manejo de excepciones [Sebesta, Cap. 14 §14.1, pp. 612–614]:

1. ¿Cómo se **especifican y clasifican** las excepciones?
2. ¿Cómo se **levanta** (raise/throw) una excepción?
3. ¿Cómo se **liga** una excepción a su handler?
4. ¿Puede **información** de la excepción pasarse al handler?
5. ¿Dónde **continúa** la ejecución después del handler?
6. ¿Se provee alguna forma de **finalización** (`finally`)?
7. ¿Pueden **definirse excepciones** por el usuario?
8. ¿Deben **declararse** excepciones predefinidas (checked vs. unchecked)?

Por qué importa esta lista

No se trata de memorizar las 8 preguntas. Se trata de entender que TypeScript, Go, Kotlin y Rust son **respuestas distintas** a estas mismas preguntas. Por ejemplo:

- **Pregunta 8 (checked vs. unchecked):** Java dice “sí” (checked exceptions obligatorias). TypeScript dice “no” (no hay checked exceptions). Kotlin dice “no” (eliminó las checked exceptions de Java). Esa única decisión cambia cómo escribís código en cada lenguaje.

Sebesta explica el mecanismo de checked exceptions en Java: “Las excepciones de clase `Error` y `RuntimeException` y sus descendientes se llaman **unchecked exceptions**. Todas las demás se llaman **checked exceptions**. Las unchecked exceptions nunca son preocupación del compilador. Sin embargo, el compilador asegura que todas las checked exceptions que un método puede lanzar están manejadas o declaradas” [Sebesta, Cap. 14 §14.3, p. 620].

TypeScript no tiene este mecanismo — `throw` puede lanzar cualquier valor, y el compilador no verifica qué excepciones puede lanzar una función. Por eso el enfoque `Result<T,E>` es más seguro en TypeScript que en Java: la exhaustividad se logra por tipos, no por declaraciones.

4.6. try / catch / finally en TypeScript

Filmina de referencia: [F-08]

El bloque `try/catch/finally` es la implementación concreta del modelo de terminación en TypeScript. Veamos la anatomía:

```
// Ejemplo completo – TypeScript
async function loadUserData(userId: string): Promise<UserProfile> {
  try {
    // Bloque try: código que puede lanzar excepciones
    const response = await fetch(`/api/users/${userId}`)

    if (!response.ok) {
      // Lanzar excepción user-defined con contexto
      throw new HttpError(response.status, `User ${userId} not found`)
    }

    return await response.json() as UserProfile
  } catch (error) {
    // Bloque catch: handler – recibe el objeto lanzado
    if (error instanceof HttpError && error.statusCode === 404) {
      throw new UserNotFoundError(userId) // re-lanzar más específica
    }
    throw error // re-lanzar si no podemos manejarla aquí
  } finally {
    // Bloque finally: SIEMPRE ejecuta (con o sin excepción)
    console.log(`loadUserData(${userId}) completado`)
  }
}
```

Flujo de control

Hay tres caminos posibles al ejecutar este bloque:

Situación	Flujo
Sin excepción	try → finally → retorno normal
Excepción capturada	try (hasta el throw) → catch → finally
Excepción no capturada	try → finally → propagación al llamador

⚠ Nota sobre finally: El bloque **finally** **siempre** ejecuta. Incluso si hacés **return** dentro del **try**, el **finally** ejecuta antes de que el valor sea retornado al caller. Si el **finally** también tiene un **return**, ese valor **eclipsa** al del **try**. Usalo con cuidado.

catch en TypeScript vs. Java

En TypeScript, el parámetro **error** del **catch** es de tipo **unknown** — el compilador te obliga a hacer un **instanceof** o narrowing antes de usarlo. En Java, podés tener múltiples cláusulas **catch** con tipos específicos:

```
// Java – múltiples catch con tipos específicos
try { ... }
catch (HttpError e) { ... }
catch (DatabaseError e) { ... }
catch (Exception e) { ... } // catch-all de último recurso
```

```
// TypeScript – un solo catch con instanceof
try { ... }
catch (error) {
  if (error instanceof HttpError) { ... }
  else if (error instanceof DatabaseError) { ... }
  else { /* unknown error */ }
}
```

Esto responde las **preguntas 5 y 6** de Sebesta: dónde continúa la ejecución (en el código después del try/catch) y si hay finalización (sí, con `finally`).

Sebesta describe el rol de `finally`: “Hay algunas situaciones en las que un proceso debe ejecutarse independientemente de si una cláusula try lanza una excepción o si la excepción se maneja en el método. Un ejemplo de tal situación es un archivo que debe cerrarse” [Sebesta, Cap. 14 §14.3.6, p. 624].

4.7. Propagación por el call stack

Filmina de referencia: [F-09]

Cuando se lanza una excepción, el runtime busca un handler **en el bloque actual**. Si no lo encuentra, la excepción **se propaga** al llamador. Y así sucesivamente, subiendo por el call stack hasta encontrar un handler o hasta que el programa termina.

Sebesta formula la regla:

“Si el bloque actual no tiene un handler, la excepción se propaga al llamador — y así sucesivamente, hasta que se encuentre un handler o el programa termine.”

— [Sebesta, *Concepts of Programming Languages*, Cap. 14 §14.2, p. 614] (traducción)

Visualización del call stack

```
main()
└─ fetchUserProfile(id)   ← sin handler → propaga
    └─ loadUserData(id)   ← catch → relanza UserNotFoundError
        └─ fetch("/api/...") ← lanza HttpError(404)
```

Flujo de propagación (hacia arriba):

```
HttpError(404)    → capturada en loadUserData
UserNotFoundError → relanzada, NO capturada en fetchUserProfile
UserNotFoundError → capturada en main
```

Pasos de propagación

- `fetch()` lanza `HttpError(404)` → busca handler en `loadUserData`.
- `loadUserData` tiene `catch` → captura, transforma, relanza `UserNotFoundError`.
- `fetchUserProfile` no tiene handler para `UserNotFoundError` → propaga.
- `main` tiene `catch (e: UserNotFoundError)` → captura y maneja.

Stack unwinding

El proceso de subir el call stack buscando un handler se llama **stack unwinding** (desenrollado de la pila). Louden & Lambert lo describen:

“El proceso de salir hacia atrás a través de las llamadas a funciones hasta el llamador durante la búsqueda de un handler se llama *call unwinding* o *stack unwinding*, siendo la pila el call stack.”

— [Louden & Lambert, *Programming Languages*, Cap. 9 §9.5, p. 431] (traducción)

Gabbrielli & Martini añaden que la propagación no es un simple salto: “Si la excepción no se maneja dentro del procedimiento que se está ejecutando actualmente, es necesario terminar el procedimiento actual y relanzar la excepción” [Gabbrielli & Martini, Cap. 7 §7.3.1, p. 282].

Información disponible en el handler

Sebesta señala que un aspecto relacionado con la ligadura excepción-handler es “si la información sobre la excepción se pone a disposición del handler” [Sebesta, Cap. 14 §14.2, p. 614]. En TypeScript, el objeto lanzado viaja con la excepción:

```
catch (error) {  
  if (error instanceof HttpError) {  
    console.log(error.statusCode) // campo custom  
    console.log(error.message)    // heredado de Error  
    console.log(error.stack)      // stack trace completo  
  }  
}
```

⚠️ **¿Qué pasa si nadie captura la excepción?** En Node.js: `UnhandledPromiseRejection` o el proceso muere. En browser: error en consola. Por eso siempre hay que tener un `catch` de último recurso en el entry point del programa.

4.8. Excepciones user-defined en TypeScript

Filmina de referencia: [F-10]

Sebesta dice que en el caso de excepciones user-defined, “el objeto lanzado podría incluir cualquier cantidad de campos de datos que podrían ser útiles en el handler” [Sebesta, Cap. 14 §14.2, p. 614]. Esto responde la **pregunta 4** de las 8 preguntas de diseño: la información de la excepción **sí** se pasa al handler, y podés incluir los campos que quieras.

Jerarquía de clases

```
// Base: extender Error siempre
class AppError extends Error {
  constructor(message: string, public readonly code: string) {
    super(message)
    this.name = this.constructor.name
    // Fix para stack trace en TypeScript con targets ES5
    if (Error.captureStackTrace) {
      Error.captureStackTrace(this, this.constructor)
    }
  }
}

// Errors específicos del dominio
class HttpError extends AppError {
  constructor(
    public readonly statusCode: number,
    message: string
  ) {
    super(message, `HTTP_${statusCode}`)
  }
}

class UserNotFoundError extends AppError {
  constructor(public readonly userId: string) {
    super(`User ${userId} not found`, 'USER_NOT_FOUND')
  }
}

class DatabaseError extends AppError {
  constructor(
    message: string,
    public readonly query?: string
  ) {
    super(message, 'DB_ERROR')
  }
}
```

¿Por qué una jerarquía?

- `catch (e instanceof AppError)` → captura **todos** los errores de dominio.
- `catch (e instanceof HttpError)` → solo HTTP.
- El campo `code` permite `switch / match` sin `instanceof` anidados.
- `this.name = this.constructor.name` hace que el error aparezca con su nombre específico en los logs (sin esto, aparece como `Error` genérico — es un quirk de TypeScript).

Error chaining — `Error.cause` (ES2022)

Desde ES2022, podés encadenar errores con `{ cause }` sin perder el contexto original:

```
// Error.cause: propiedad estándar desde ES2022 (TC39 Stage 4 – Node.js ≥ 16.9)
async function loadUserData(userId: string): Promise<User> {
  try {
    const res = await fetch(`/api/users/${userId}`)
    if (!res.ok) throw new HttpError(res.status, `HTTP ${res.status}`)
    return res.json()
  } catch (e) {
    // Sintaxis ES2022: segundo argumento { cause } preserva el error original
    throw new Error('Failed to load user data', { cause: e })
  }
}

// Inspección de la cadena de causas
try {
  await loadUserData('42')
} catch (err) {
  console.error('Error principal:', err.message) // 'Failed to load user data'
  console.error('Causado por:', err.cause) // HttpError original preservado
  console.error('Mensaje origen:', err.cause?.message) // 'HTTP 404'
}
```

Antes de ES2022: se usaba `wrapErr.cause = originalErr` (no estándar, sin soporte en herramientas de debug). Desde ES2022 el constructor `new Error(msg, { cause })` es parte del estándar ECMAScript — DevTools y stack tracers inspeccionan automáticamente la cadena. Mismo patrón que `fmt.Errorf("...: %w", err)` en Go.

4.9. El enfoque funcional: Result<T,E>

Filmina de referencia: [F-11]

Motivación

`throw` es un **efecto secundario**: rompe el flujo de ejecución de una manera que es difícil de razonar en programación funcional. Si una función puede lanzar una excepción, su firma no lo dice — el llamador no sabe qué excepciones esperar. La alternativa: retornar un **tipo suma** que codifica éxito o error como un valor normal.

Definición

```
// Definición de Result<T,E>
type Result<T, E extends Error> =
  | { ok: true; value: T }
  | { ok: false; error: E }

// Helpers
const ok = <T>(value: T): Result<T, never> => ({ ok: true, value })
const err = <E extends Error>(error: E): Result<never, E> => ({ ok: false, error })
```

Esto es una **discriminated union**: el campo `ok` es el discriminante. Si `ok` es `true`, TypeScript sabe que hay un `value`. Si `ok` es `false`, sabe que hay un `error`.

Implementación

```
async function fetchUser(id: string): Promise<Result<User, HttpError>> {
  const res = await fetch(`/api/users/${id}`)
  if (!res.ok) return err(new HttpError(res.status, `HTTP ${res.status}`))
  return ok(await res.json() as User)
}
```

Fijate que **no hay** `throw`. Si algo falla, retornamos un valor. El flujo de control nunca se rompe.

Uso — exhaustivo por discriminated union

```
const result = await fetchUser("42")

if (result.ok) {
  // TypeScript sabe que result.value: User
  console.log(result.value.name)
} else {
  // TypeScript sabe que result.error: HttpError
  console.error(`Error ${result.error.statusCode}: ${result.error.message}`)
}
```

El **narrowing** de TypeScript hace que si `result.ok` es `true`, el compilador sabe que `result.value` es `User`. No podés acceder a `result.error` en ese branch — el compilador te lo impide. Esto es **exhaustividad verificada en compilación**.

Composición con map/flatMap

```
const user = await fetchUser("42")
  .then(r => r.ok ? ok(r.value.email.toLowerCase()) : r)

// 0 con librería como neverthrow / fp-ts
```

Acá es donde `Result` brilla: podés encadenar operaciones sin `try/catch` anidados. Si ya conocés las mónadas del Tema 5, esto es `map / flatMap` sobre `Either`.

Conexión curricular: `Result<T, E>` es la mónada `Either`. En Haskell es `Either a b`. En fp-ts es `Either`. La idea es la misma. [Conexión: Tema 5 — Mónadas]

Gabbrielli & Martini observan que los tipos suma permiten que una excepción “no sea un evento anónimo — tiene un nombre, a menudo un valor en uno de los tipos del lenguaje” [Gabbrielli & Martini, Cap. 7 §7.3.1, p. 282]. El enfoque funcional lleva esta idea al extremo: el error es un valor tipado, no un efecto secundario.

4.10. Comparativa multi-lenguaje

Filminas de referencia: [F-12], [F-13], [F-14], [F-15]

Cada lenguaje responde las 8 preguntas de Sebesta de forma distinta. Veamos cómo.

4.10.1. Go: errors as values

Filmina de referencia: [F-12]

Go **no tiene** `try/catch` — fue una decisión deliberada de los diseñadores (Rob Pike, Ken Thompson, Robert Griesemer). Los errores son **valores** de retorno normales, tipo `error` (interfaz). `panic` / `recover` existen pero se usan solo para casos excepcionales reales.

```
// Go - retorno múltiple: (resultado, error)
func fetchUser(id string) (*User, error) {
    resp, err := http.Get("/api/users/" + id)
    if err != nil {
        // Wrap error con contexto usando %w (Go 1.13+)
        return nil, fmt.Errorf("fetchUser: %w", err)
    }
    defer resp.Body.Close() // defer = equivalente a finally

    if resp.StatusCode == 404 {
        return nil, &UserNotFoundError{ID: id}
    }

    var u User
    if err := json.NewDecoder(resp.Body).Decode(&u); err != nil {
        return nil, fmt.Errorf("decode user: %w", err)
    }
    return &u, nil
}

// Caller - OBLIGADO a manejar el error (o ignorarlo explícitamente con _)
user, err := fetchUser("42")
if err != nil {
    var notFound *UserNotFoundError
    if errors.As(err, &notFound) { // errors.As unwrap la cadena
        log.Printf("usuario %s no existe", notFound.ID)
        return
    }
    return fmt.Errorf("error inesperado: %w", err)
}
```

Características clave:

- `defer` es el `finally` de Go — ejecuta cuando la función retorna, haya error o no.
- `%w` en `fmt.Errorf` envuelve errores (Go 1.13+). `errors.As` y `errors.Is` los desenvuelven.
- Go 1.20 (2023): `errors.Join` une varios errores en uno solo — útil en validaciones que pueden fallar en múltiples puntos.

```
// Go 1.20 (2023): errors.Join une varios errores en uno solo
func validateUser(u User) error {
    var errs []error
    if u.Name == "" {
        errs = append(errs, errors.New("name is required"))
    }
    if u.Email == "" {
        errs = append(errs, errors.New("email is required"))
    }
    if u.Age < 0 {
        errs = append(errs, errors.New("age must be non-negative"))
    }
    return errors.Join(errs...) // nil si errs está vacío
}
```

Ventajas	Inconvenientes
Explícito: el error está visible en la firma	Verboso: <code>if err != nil</code> repetido en cada llamada
Sin stack unwinding — más predecible en performance	Fácil de ignorar: <code>user, _ := fetchUser("42")</code> silencia el error

4.10.2. Kotlin: sealed classes y try-expression

Filmina de referencia: [F-13]

Kotlin eliminó las checked exceptions de Java y ofrece `sealed class` para errores tipados con exhaustividad verificada por el compilador.

```
// Sealed class = union type exhaustiva y verificada por el compilador
sealed class UserResult {
    data class Success(val user: User) : UserResult()
    data class NotFound(val id: String) : UserResult()
    data class NetworkError(val statusCode: Int) : UserResult()
}

// try en Kotlin es una EXPRESIÓN (retorna valor)
fun fetchUser(id: String): UserResult = try {
    val resp = httpClient.get("/api/users/$id")
    when (resp.status) {
        200 -> UserResult.Success(resp.body<User>())
        404 -> UserResult.NotFound(id)
        else -> UserResult.NetworkError(resp.status)
    }
} catch (e: IOException) {
    UserResult.NetworkError(-1)
}
```

Uso — `when` es exhaustivo:

```
val result = fetchUser("42")
when (result) {
    is UserResult.Success -> println(result.user.name)
    is UserResult.NotFound -> println("No encontrado: ${result.id}")
    is UserResult.NetworkError -> println("Error HTTP ${result.statusCode}")
    // No hay else — el compilador verifica que todos los casos estén cubiertos
}
```

Diferencias con TypeScript:

- Kotlin: **sin checked exceptions** (a diferencia de Java).
- `try` como **expresión** → retorna valor, se integra naturalmente con FP.
- `sealed class` = discriminated union del compilador (vs. union types estructurales de TS).

4.10.3. Rust: `Result<T,E>` y el operador `?`

Filmina de referencia: [F-14]

Rust garantiza que **no existen excepciones en runtime** (excepto `panic!` explícito). Toda operación fallable retorna `Result<T, E>` — el tipo fuerza el manejo en el compilador. El operador `?` azucariza la propagación.

```
// Definir errores del dominio
#[derive(Debug)]
enum FetchError {
    Network(request::Error),
    NotFound(String),
    Decode(serde_json::Error),
}

// impl From<request::Error> for FetchError { ... } → permite usar ?

async fn fetch_user(id: &str) -> Result<User, FetchError> {
    let resp = request::get(format!("/api/users/{id}"))
        .await
        .map_err(FetchError::Network)?; // ? = propagación implícita

    if resp.status() == 404 {
        return Err(FetchError::NotFound(id.to_string()));
    }

    resp.json::<User>()
        .await
        .map_err(FetchError::Decode)
}
```

El operador `?` desazucarado:

```
// Esto:
let resp = request::get(url).await?;

// Es equivalente a:
let resp = match request::get(url).await {
    Ok(val) => val,
    Err(e)  => return Err(e.into()),
};
```

Ventajas clave:

- Exhaustividad **garantizada por el compilador** — no podés ignorar un error.
- Sin runtime overhead de stack unwinding.
- `?` hace el código conciso manteniendo la seguridad.
- `Drop` trait = RAII → limpieza automática sin `finally`.

4.10.4. Tabla comparativa: imperativo vs. funcional, 4 lenguajes

Filmina de referencia: [F-15]

Aspecto	TS throw/catch	TS Result<T, E>	Go (T, error)	Kotlin sealed	Rust Result<T, E>
Mecanismo	Excepción (throw)	Valor de retorno	Valor de retorno	Expresión (try)	Valor de retorno
Flujo	Ruptura de stack	Normal	Normal	Normal	Normal
Exhaustividad	Runtime	Union type TS	Ninguna (ignorable)	Compilador (when)	Compilador (match)
Composabilidad	Difícil en FP	map/flatMap	Secuencial if/err	Fluida (expr)	? operator
Stack unwinding	Sí (costoso)	No	No	Sí (si lanza)	No
Limpieza recursos	finally	Manual	defer	finally / use	Drop (RAII)
Verbosidad	Bajo	Medio	Alto (if err != nil)	Bajo	Bajo (con ?)
async/Promise	Natural	Promise<Result>	goroutines	suspend	async/await

Regla de selección

- **TS imperativo** (throw): código OO, async/await, equipos con background Java/JS.
- **TS funcional** (Result): pipelines de datos, código FP, integración con sistemas agénticos.
- **Go**: sistemas de bajo nivel, microservicios, cuando la legibilidad explícita importa.
- **Kotlin**: Android, backend JVM donde la exhaustividad en compile time importa.
- **Rust**: sistemas, WASM, cuando la seguridad en memoria es crítica.

⚠ ¿Podemos mezclar throw y Result en el mismo proyecto? Sí, y es lo más común.

Generalmente: throw para errores inesperados de sistema, Result para errores de negocio esperados.

4.11. Excepciones en programación agéntica

Filmina de referencia: [F-16]

Este sub-tema no está cubierto en Sebesta (su libro es de 2019). Es el contexto de la cátedra y de la industria actual.

El problema: errores en cascada (compounding errors)

Un agente de IA encadena múltiples llamadas a tools (fetch data → process → store). Si fetch falla silenciosamente (catch {} vacío), el agente procesa **datos inconsistentes**. Los pasos siguientes

actúan sobre un estado corrupto → **el agente alucina resultados**.

“Los agentes autónomos tienen mayor costo y potencial de **errores en cascada** — cada error no manejado se amplifica en los pasos siguientes.”

— [Anthropic, “Building effective agents”, dic. 2024]

Validación del estado en cada paso

Anthropic recomienda que el agente obtenga **verdad del terreno** (*ground truth*) del entorno en cada paso — resultados de tool calls, ejecución de código — para evaluar su progreso. El patrón clave: **nunca asumir que un paso anterior fue exitoso** sin verificar su resultado.

Las **condiciones de parada** (*stopping conditions*) — máximo de intentos, tiempo límite — son el mecanismo de control ante errores irreversibles.

Interfaz Agente-Computadora (ACI)

Anthropic plantea que hay que invertir en el diseño de tools tanto como en el diseño de prompts. “Poka-yoke” tus tools: diseñalas para que sea **difícil cometer errores** — ej: requerir rutas absolutas en lugar de relativas. Los contratos de error de cada tool son parte del ACI: qué retorna si falla, en qué casos es reintentable.

MCP Protocol y errores tipados

El protocolo MCP (Anthropic 2024) especifica que las tools deben retornar errores estructurados:

```
// MCP spec (Anthropic 2024): las tools deben retornar errores estructurados
interface ToolResult {
  content: Array<{ type: "text" | "image", text?: string }>
  isError?: boolean // true = el agente sabe que falló
}

// Tool que usa Result internamente pero expone MCP
async function eduSearchTool(query: string): Promise<ToolResult> {
  const result = await searchKnowledge(query) // retorna Result<T,E>

  if (!result.ok) {
    return {
      isError: true,
      content: [{ type: "text", text: `Error: ${result.error.message}` }]
    }
  }
  return {
    content: [{ type: "text", text: JSON.stringify(result.value) }]
  }
}
```

Fijate que esto es **exactamente el enfoque funcional aplicado**: la tool usa `Result` internamente y expone `isError: true` en la interfaz MCP. El agente sabe que la tool falló y puede decidir reintentar, usar una tool alternativa, o abortar.

Reintento con espera exponencial — patrón agéntico

```
async function withRetry<T>(  
  fn: () => Promise<Result<T, Error>>,  
  maxAttempts = 3 // condición de parada: máximo de intentos  
) : Promise<Result<T, Error>> {  
  for (let i = 0; i < maxAttempts; i++) {  
    const result = await fn()  
    if (result.ok) return result  
    if (!isRetryable(result.error)) return result // error irrecoverable  
    await sleep(2 ** i * 200) // espera exponencial  
  }  
  return { ok: false, error: new MaxRetriesError(maxAttempts) }  
}
```

La diferencia entre un error de red (retryable) y un error de autenticación (no retryable) importa. Por eso los **tipos de error** importan: `NetworkTimeoutError` → retry; `AuthenticationError` → no retry.

Conexión con esta cátedra

- `publish_loop.py` de EDU usa exactamente este patrón: reintento hasta 3 veces, registra errores en `error-registry.jsonl`.
- `edu-mcp-server` retorna `isError: true` cuando ChromaDB falla — el agente sabe cómo reaccionar.

5. Ejemplos Trabajados

Ejemplo 1: Trazar propagación por el call stack

Enunciado: Dado el siguiente código TypeScript, trazá el flujo de propagación de la excepción. ¿Qué se imprime en consola? ¿Qué función captura la excepción final?

```

class ValidationError extends Error {
  constructor(public field: string) {
    super(`Invalid field: ${field}`)
    this.name = 'ValidationError'
  }
}

function validateAge(age: number): void {
  if (age < 0) throw new ValidationError('age')
  if (age > 150) throw new ValidationError('age')
}

function processUser(user: { name: string; age: number }): string {
  validateAge(user.age)
  return `User ${user.name} processed`
}

function handleRequest(req: { name: string; age: number }): string {
  try {
    return processUser(req)
  } catch (e) {
    if (e instanceof ValidationError) {
      return `Validation failed: ${e.field}`
    }
    throw e
  }
}

// Llamada:
const result = handleRequest({ name: "Ana", age: -5 })
console.log(result)

```

Resolución paso a paso:

1. `handleRequest({ name: "Ana", age: -5 })` se invoca.
2. Dentro del `try`, se llama a `processUser({ name: "Ana", age: -5 })`.
3. `processUser` llama a `validateAge(-5)`.
4. `validateAge` evalúa `age < 0` → `true` → lanza `ValidationError('age')`.
5. `validateAge` no tiene handler → la excepción se propaga a `processUser`.
6. `processUser` no tiene handler → la excepción se propaga a `handleRequest`.
7. `handleRequest` tiene `catch (e)` → captura la excepción.
8. `e instanceof ValidationError` → `true` → retorna `"Validation failed: age"`.
9. `console.log(result)` imprime: `Validation failed: age`.

Call stack visualizado:

```

handleRequest({ name: "Ana", age: -5 }) ← catch → captura ValidationError
├─ processUser({ name: "Ana", age: -5 }) ← sin handler → propaga
│   └─ validateAge(-5) ← lanza ValidationError('age')

```

Respuesta: Se imprime `Validation failed: age`. La excepción es capturada en `handleRequest`.

Ejemplo 2: Convertir try/catch a Result<T,E>

Enunciado: La siguiente función usa `try/catch` (enfoque imperativo). Reescribirla usando `Result<T,E>` (enfoque funcional).

```
// Versión imperativa
function parseConfig(raw: string): Config {
  try {
    const parsed = JSON.parse(raw)
    if (!parsed.host || !parsed.port) {
      throw new Error("Missing required fields: host, port")
    }
    if (typeof parsed.port !== 'number' || parsed.port < 0 || parsed.port > 65535) {
      throw new Error("Invalid port number")
    }
    return parsed as Config
  } catch (e) {
    if (e instanceof SyntaxError) {
      throw new Error(`JSON syntax error: ${e.message}`)
    }
    throw e
  }
}
```

Resolución paso a paso:

Paso 1 — Definir los tipos de error del dominio:

```
class ConfigError extends Error {
  constructor(message: string, public readonly kind: 'syntax' | 'missing_fields' | 'invalid_port') {
    super(message)
    this.name = 'ConfigError'
  }
}

type Result<T, E extends Error = Error> =
  | { ok: true; value: T }
  | { ok: false; error: E }

const ok = <T>(value: T): Result<T, never> => ({ ok: true, value })
const err = <E extends Error>(error: E): Result<never, E> => ({ ok: false, error })
```

Paso 2 — Reescribir la función sin `throw`:

```
// Versión funcional
function parseConfig(raw: string): Result<Config, ConfigError> {
  let parsed: unknown
  try {
    parsed = JSON.parse(raw)
  } catch (e) {
    if (e instanceof SyntaxError) {
      return err(new ConfigError(`JSON syntax error: ${e.message}`, 'syntax'))
    }
    return err(new ConfigError(`Unexpected error: ${e}`, 'syntax'))
  }

  const obj = parsed as Record<string, unknown>

  if (!obj.host || !obj.port) {
    return err(new ConfigError("Missing required fields: host, port", 'missing_fields'))
  }

  if (typeof obj.port !== 'number' || obj.port < 0 || obj.port > 65535) {
    return err(new ConfigError("Invalid port number", 'invalid_port'))
  }

  return ok(parsed as Config)
}
```

Paso 3 — Uso con exhaustividad:

```
const result = parseConfig('{ "host": "localhost", "port": 8080 }')

if (result.ok) {
  console.log(`Config loaded: ${result.value.host}:${result.value.port}`)
} else {
  switch (result.error.kind) {
    case 'syntax': console.error(`Syntax error: ${result.error.message}`); break
    case 'missing_fields': console.error(`Missing fields: ${result.error.message}`); break
    case 'invalid_port': console.error(`Bad port: ${result.error.message}`); break
  }
}
```

Diferencias clave:

- La versión imperativa **lanza** excepciones que el llamador no sabe que existen (no hay en la firma).
- La versión funcional **retorna** `Result<Config, ConfigError>` — el llamador sabe exactamente qué puede fallar.
- El `switch` sobre `result.error.kind` es exhaustivo: el compilador puede verificar que todos los casos están cubiertos.
- El `try/catch` que queda es solo para `JSON.parse` (que es una función de biblioteca que lanza `SyntaxError` — no podemos cambiarla). El resto del código no usa `throw`.

Ejemplo 3: Análisis de excepciones en un sistema agéntico

Enunciado: Un agente de IA tiene un pipeline de 3 pasos: (1) buscar datos en una API, (2) procesar los datos con un modelo, (3) guardar el resultado en una base de datos. El siguiente código tiene un bug grave de manejo de errores. Identifícalo y proponé una solución con `Result<T,E>`.

```
// CÓDIGO CON BUG — no usar en producción
async function agentPipeline(query: string): Promise<string> {
  // Paso 1: buscar datos
  let data: any
  try {
    const res = await fetch(`/api/search?q=${query}`)
    data = await res.json()
  } catch (e) {
    console.log("Search failed, continuing with empty data")
    data = null // ← BUG: el agente sigue con datos vacíos
  }

  // Paso 2: procesar con modelo
  const processed = await llmProcess(data) // ← recibe null, alucina

  // Paso 3: guardar resultado
  await db.save(processed)

  return processed
}
```

Análisis del bug:

El problema es **context poisoning**: si el paso 1 falla, el `catch` vacío silencia el error y asigna `data = null`. El paso 2 recibe `null` y el modelo LLM **alucina** un resultado a partir de nada. El paso 3 guarda ese resultado alucinado en la base de datos. El agente opera con total confianza sobre datos corruptos.

Esto es exactamente lo que Anthropic describe: “errores en cascada — cada error no manejado se amplifica en los pasos siguientes” [Anthropic, “Building effective agents”, dic. 2024].

Solución con Result<T,E>:

```
class SearchError extends Error { constructor(public query: string) { super(`Search failed: ${query}`) } }
class ProcessError extends Error { constructor(public reason: string) { super(`Process failed: ${reason}`) } }
class SaveError extends Error { constructor(public data: unknown) { super(`Save failed`) } }

type Result<T, E extends Error = Error> =
  | { ok: true; value: T }
  | { ok: false; error: E }

const ok = <T>(value: T): Result<T, never> => ({ ok: true, value })
const err = <E extends Error>(error: E): Result<never, E> => ({ ok: false, error })

// Paso 1: buscar datos – retorna Result
async function searchData(query: string): Promise<Result<unknown, SearchError>> {
  try {
    const res = await fetch(`/api/search?q=${query}`)
    if (!res.ok) return err(new SearchError(query))
    return ok(await res.json())
  } catch (e) {
    return err(new SearchError(query))
  }
}

// Paso 2: procesar – solo se ejecuta si el paso 1 fue exitoso
async function processData(data: unknown): Promise<Result<string, ProcessError>> {
  try {
    const result = await llmProcess(data)
    return ok(result)
  } catch (e) {
    return err(new ProcessError(String(e)))
  }
}

// Paso 3: guardar – solo se ejecuta si el paso 2 fue exitoso
async function saveResult(data: string): Promise<Result<void, SaveError>> {
  try {
    await db.save(data)
    return ok(undefined)
  } catch (e) {
    return err(new SaveError(data))
  }
}

// Pipeline con propagación explícita de errores
async function agentPipeline(query: string): Promise<Result<string, Error>> {
  // Paso 1
  const searchResult = await searchData(query)
  if (!searchResult.ok) return searchResult // propaga el error, NO continúa

  // Paso 2 – solo se ejecuta si el paso 1 fue ok
  const processResult = await processData(searchResult.value)
  if (!processResult.ok) return processResult

  // Paso 3 – solo se ejecuta si el paso 2 fue ok
  const saveResult = await saveResult(processResult.value)
  if (!saveResult.ok) return saveResult

  return ok(processResult.value)
}
```

Por qué esta solución es mejor:

1. **No hay context poisoning:** si el paso 1 falla, el pipeline **se detiene** y retorna el error. El paso 2 nunca ejecuta con datos corruptos.
2. **El llamador sabe qué puede fallar:** la firma `Promise<Result<string, Error>>` dice explícitamente que puede retornar un error.
3. **Cada paso es independiente:** si querés agregar reintentos al paso 1 (que es el más propenso a fallar por red), podés envolver `searchData` con `withRetry` sin tocar el resto del pipeline.
4. **El error preserva contexto:** `SearchError` sabe qué query falló, `ProcessError` sabe la razón, `SaveError` sabe qué dato no se pudo guardar.

6. Puntos Clave (Cheat-Sheet)

Las 8 preguntas de diseño de Sebesta (§14.1, pp. 612–614)

#	Pregunta	TS	Go	Kotlin	Rust
1	¿Cómo se especifican?	Objetos Error	Interfaz <code>error</code>	Sealed class	Enum
2	¿Cómo se levanta?	<code>throw</code>	Retorno de <code>error</code>	<code>throw</code> / retorno	Retorno de <code>Err</code>
3	¿Cómo se liga al handler?	<code>catch</code> con <code>instanceof</code>	<code>if err != nil</code>	<code>when</code> exhaustivo	<code>match</code> exhaustivo
4	¿Info al handler?	Sí (campos custom)	Sí (interfaz)	Sí (data class)	Sí (enum variant)
5	¿Dónde continúa?	Después del try	Después del if	Después del when	Después del match
6	¿Finalización?	<code>finally</code>	<code>defer</code>	<code>finally</code> / <code>use</code>	Drop (RAII)
7	¿User-defined?	Sí (extender Error)	Sí (struct + <code>error()</code>)	Sí (sealed subclass)	Sí (enum variant)
8	¿Checked?	No	No	No	N/A (no hay throw)

Los 5 puntos que no podés olvidar

1. **Excepción = evento anómalo en runtime** — no es un bug, es una condición que el código normal no maneja [Sebesta §14.1, p. 611].
2. **El modelo de terminación domina** — al lanzar, el scope se termina. No hay vuelta al punto del raise [Sebesta §14.2.3, p. 612].
3. **Propagación sube el stack** — si no hay handler local, sube al llamador hasta encontrar uno o terminar el programa [Sebesta §14.2, p. 614]. El proceso se llama *stack unwinding* [Louden §9.5, p. 431].
4. **Dos filosofías de diseño:**
 - **Imperativo** (`throw`): expresivo, natural en async, estándar OO.
 - **Funcional** (`Result`): composable, exhaustivo en tipos, esencial en sistemas agénticos.
5. **En programación agéntica:** los errores no manejados envenenan el contexto — siempre tipar y propagar errores en tools MCP [Anthropic, 2024].

Terminación vs. Reanudación — resumen

	Terminación	Reanudación
Al lanzar	El scope termina	El scope se pausa
Después del handler	No vuelve al raise	Vuelve al punto del raise
Lenguajes	Todos los modernos	PL/I (legacy)
Por qué domina	Simple, predecible, bajo acoplamiento	Requiere conocimiento del contexto interno del caller

Decisión de diseño por lenguaje — resumen

Lenguaje	Enfoque	Exhaustividad	Limpieza
TypeScript (imperativo)	<code>throw/catch</code>	Runtime	<code>finally</code>
TypeScript (funcional)	<code>Result<T,E></code>	Union type	Manual
Go	<code>(T, error)</code>	Ninguna (ignorable)	<code>defer</code>
Kotlin	<code>sealed class</code>	Compilador (<code>when</code>)	<code>finally</code> / <code>use</code>
Rust	<code>Result<T,E></code>	Compilador (<code>match</code>)	Drop (RAII)

7. Autoevaluación

Las siguientes 10 preguntas cubren los distintos niveles de la taxonomía de Bloom. Intentá resolverlas sin consultar la guía. Las respuestas están en bloques desplegados.

Pregunta 1 (Recordar)

¿Cuál es la definición de excepción según Sebesta §14.1?

► Respuesta

Pregunta 2 (Recordar)

Nombrá los cuatro componentes del mecanismo de excepciones identificados por Sebesta.

► Respuesta

Pregunta 3 (Comprender)

¿Cuál es la diferencia entre el modelo de terminación y el de reanudación? ¿Por qué la terminación domina en los lenguajes modernos?

► Respuesta

Pregunta 4 (Comprender)

Explicá qué es el *stack unwinding* y por qué la propagación de excepciones no es un simple salto.

► Respuesta

Pregunta 5 (Aplicar)

Escribí un bloque `try/catch/finally` en TypeScript que: (a) intente parsear un string como JSON, (b) capture `SyntaxError` y lo re-lance como `ParseError` con el mensaje original, © en el `finally`, imprima "Parse attempt finished" siempre.

► Respuesta

Pregunta 6 (Aplicar)

Implementá la función `divide(a: number, b: number): Result<number, string>` usando el tipo `Result<T, E>`. Si `b` es cero, retorná un error; si no, retorná el cociente.

► Respuesta

Pregunta 7 (Analizar)

Compará cómo responden Go y Rust a la pregunta 8 de Sebesta (checked vs. unchecked exceptions). ¿Por qué ninguno de los dos tiene checked exceptions?

► Respuesta

Pregunta 8 (Analizar)

Dado el siguiente código, explicá qué imprime y por qué. ¿Qué problema demuestra?

```
async function step1(): Promise<string> {
  throw new Error("Network failed")
}

async function step2(data: string): Promise<string> {
  return data.toUpperCase()
}

async function pipeline(): Promise<string> {
  let data: string
  try {
    data = await step1()
  } catch {
    data = "" // silenciar error
  }
  return await step2(data)
}

console.log(await pipeline())
```

► Respuesta

Pregunta 9 (Evaluar)

En un proyecto TypeScript que combina código OO (clases con herencia) y pipelines de procesamiento de datos funcionales, ¿cuándo usarías `throw` y cuándo `Result<T, E>`? Justificá tu decisión.

► Respuesta

Pregunta 10 (Crear / Identificar)

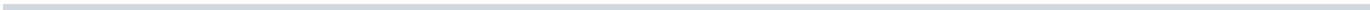
Diseñá una interfaz MCP `ToolResult` para una tool que busca en una base de conocimiento. La tool debe: (a) retornar resultados estructurados si la búsqueda es exitosa, (b) retornar `isError: true` si la query está vacía o si la base de datos no responde, © incluir un campo `retryable` para que el agente sepa si puede reintentar.

► Respuesta

8. Glosario

Término	Definición
Excepción	Evento inusual, erróneo o no, detectable por hardware o software, que puede requerir procesamiento especial [Sebesta §14.1, p. 611].
Exception handler	Código que responde a una excepción. Se busca en el call stack desde el bloque actual hacia arriba.
Raise / throw	Disparar una excepción. En TypeScript: <code>throw new Error(...)</code> . En Go: retornar un valor <code>error</code> . En Rust: retornar <code>Err(...)</code> .
Catch	Capturar una excepción en un handler. En TypeScript: bloque <code>catch (error)</code> .
try / catch / finally	Constructo de manejo de excepciones. <code>try</code> envuelve código que puede fallar, <code>catch</code> captura la excepción, <code>finally</code> ejecuta siempre (con o sin excepción).
Propagación	Mecanismo por el cual una excepción no capturada sube al llamador en el call stack, y así sucesivamente, hasta encontrar un handler o terminar el programa [Sebesta §14.2, p. 614].
Stack unwinding	Proceso de salir hacia atrás a través de las llamadas a funciones durante la búsqueda de un handler. Cada nivel terminado libera sus recursos [Louden §9.5, p. 431].
Terminación	Modelo de continuación donde el scope que generó la excepción se termina — el handler toma control y no se vuelve al punto del raise. Modelo dominante en lenguajes modernos [Sebesta §14.2.3, p. 612].
Reanudación	Modelo de continuación donde el handler ejecuta y devuelve control al punto del raise. Usado por PL/I (legacy). Requiere alto acoplamiento entre handler y caller.
Checked exception	Excepción que el compilador obliga a declarar o manejar. Existe en Java (clases que descienden de <code>Throwable</code> pero no de <code>Error</code> ni <code>RuntimeException</code>). No existe en TypeScript, Kotlin, Go ni Rust [Sebesta §14.3, p. 620].
Unchecked exception	Excepción que no requiere declaración ni manejo obligatorio. En Java: <code>Error</code> , <code>RuntimeException</code> y sus descendientes. En TypeScript: todas las excepciones son unchecked.
Excepción user-defined	Excepción definida por el programador, típicamente extendiendo <code>Error</code> (TS/JS) o creando un struct que implementa <code>error</code> (Go) o un variant de enum (Rust). Puede incluir campos custom útiles para el handler [Sebesta §14.2, p. 614].
Result<T,E>	Tipo suma que codifica éxito (<code>Ok<T></code>) o error (<code>Err<E></code>). Implementado como discriminated union en TypeScript, como enum en Rust, como sealed class en Kotlin. Es la mónada Either.
Option	Tipo suma que codifica presencia (<code>Some<T></code>) o ausencia (<code>None</code>). En TypeScript se aproxima con <code>T</code>
Sealed class	Jerarquía de clases restringida donde el compilador conoce todos los subtipos posibles. En Kotlin, permite exhaustividad en <code>when</code> . Equivalente a un enum con datos o una discriminated union.

Término	Definición
Errors as values	Filosofía de diseño donde los errores son valores de retorno normales, no excepciones que rompen el flujo. Adoptada por Go (<code>(T, error)</code>) y Rust (<code>Result<T, E></code>).
Discriminated union	Union type donde un campo común (el discriminante, ej: <code>ok: boolean</code>) le permite al compilador saber qué variante es. Base del patrón <code>Result<T, E></code> en TypeScript.
Error.cause	Propiedad estándar desde ES2022 que permite encadenar errores sin perder el contexto original. Equivalente a <code>%w</code> en <code>fmt.Errorf</code> de Go.
Stack unwinding cost	Overhead de performance al desenrollar el call stack durante la propagación de una excepción. Los enfoques funcionales (<code>Result</code>) no tienen este costo.
RAII	<i>Resource Acquisition Is Initialization</i> . Patrón donde la limpieza de recursos se asocia al ciclo de vida del tipo (Drop trait en Rust). Elimina la necesidad de <code>finally</code> .
Context poisoning	En programación agéntica, cuando un error no manejado en un paso intermedio “envenena” el contexto: los pasos siguientes operan sobre datos corruptos sin saberlo [Anthropic, 2024].
ACI	<i>Agent-Computer Interface</i> . Diseño de las tools que un agente usa, incluyendo contratos de error. Anthropic recomienda invertir en ACI tanto como en prompts.
Poka-yoke	Diseñar tools para que sea difícil cometer errores (ej: requerir rutas absolutas en lugar de relativas). Concepto de manufactura japonesa aplicado al diseño de tools agénticas.
Continuador	Función o constructo que representa “qué hacer después”. En el contexto de excepciones, el modelo de continuación define qué pasa después de que el handler ejecuta.



9. Referencias

Bibliografía primaria

1. **Sebesta, R. W.** (2019). *Concepts of Programming Languages* (11ª ed.). Pearson. Cap. 14: "Exception Handling and Event Handling", pp. 611–646.
 - §14.1 — Introducción y preguntas de diseño (pp. 611–614)
 - §14.2 — Excepciones en lenguajes comunes (pp. 614–630)
 - §14.2.3 — Continuación: terminación vs. reanudación (p. 612)
 - §14.3 — Excepciones en Java (pp. 618–626)
 - §14.3.6 — La cláusula `finally` (p. 624)
2. **Gabbrielli, M. & Martini, S.** (2023). *Programming Languages: Principles and Paradigms* (2ª ed.). Springer. Cap. 7 §7.3.1: "Implementing Exceptions", pp. 136–282.
 - Manejo con terminación (p. 282)
 - Propagación de excepciones (p. 282)
 - Excepciones como eventos con nombre (p. 282)
3. **Louden, K. C. & Lambert, K. A.** (2012). *Programming Languages: Principles and Practices* (4ª ed.). Course Technology. Cap. 9 §9.5: "Exception Handling", pp. 406–447.
 - Callbacks de error pre-try/catch (p. 425)
 - Stack unwinding (p. 431)
 - Implementación de entornos de excepciones (pp. 477–478)

Referencias secundarias

4. **Anthropic** (2024). "Building effective agents". Publicación técnica, diciembre 2024.
 - Errores en cascada en sistemas agénticos
 - Validación del estado (ground truth) en cada paso
 - Interfaz Agente-Computadora (ACI) y poka-yoke
5. **ECMA International** (2022). ECMAScript 2022 Language Specification (ES2022).
 - `Error.cause` — TC39 Stage 4, Node.js ≥ 16.9
6. **Go Team** (2023). Go 1.20 Release Notes.
 - `errors.Join` — unión de múltiples errores
 - `fmt.Errorf` con múltiples `%w`

Conexiones curriculares

- ← **Tema 11** (Estructuras de Control): las excepciones son un mecanismo de control de flujo no lineal.
- ← **Tema 5** (Mónadas en TypeScript): `Result<T,E>` es la mónada Either — `map` / `flatMap` ya conocidos.
- → **Tema 13** (Abstracción Procedural y Modularidad): cómo las excepciones cruzan límites de módulos.

Lecturas recomendadas (opcionales)

- **Pike, R.** (2015). "Errors are values". The Go Blog. — Filosofía de diseño de errores en Go.
- **fp-ts** (documentación). `Either` — implementación de `Result` en TypeScript con mónadas completas.
- **neverthrow** (documentación). Railway Oriented Programming en TypeScript.

 Si un alumno puede estudiarlo solo, lo hicimos bien.

Guía generada por Dra. Sofía (study-guide-writer) · Pipeline v3 · Nivel densidad 2 · 2026-06-28